In programming, choosing the right data structure can make a difference[14]. That's not always enough, because you also need to use it correctly.

Suppose, for instance, that we have a static set of elements, available from the beginning. We could use a max-heap, insert all `n` elements, and then extract the largest `k` of them.

We would need `O(n)` extra space for the heap, and then use `heapify` to create it from the full set in linear time, `O(n)`. Then we would call top `k` times, with a cost of `O(log(n))` for each call.

The total cost for this solution would be `O(n+k*log(n))` comparisons and swaps.

That's already better than the naïve solutions… but, if you think about it, if[15] `n>>k`, we are creating a huge heap just to extract a few elements. That certainly sounds wasteful.

2.7.2 … and the right use

So, our goal should be having a small heap with `k` elements. Using a max-heap, though, doesn't really work anymore. Let's see why with an example.

Suppose we want to find the largest 3 of the following numbers: 2, 4, 1, 3, 7, 6.

We add the first 3 and we have the following max-heap: [4, 2, 1]. Now we proceed to the next number in the list, and it's 3. It's larger than the 2 out of three elements currently in the heap, but we have no way to know, because we only can